



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

Universidad Autónoma de Nuevo León

Facultad de Ingeniería Mecánica y Eléctrica

Lenguajes de Programación y Lab

Actividad Fundamental 7: Reporte de Lenguaje Lógico

Unidad 3

Semestre: 3ro

Equipo: 04

Grupo: 025

Salón: 3109

Periodo: Enero – Junio

Docente: Ing. Selene Guadalupe Pinal Torres

Hora: M4-M6

Nombre	Matricula	Carrera
Ruedas Vazquez Jordan Emanuel	2103677	ITS
Enríquez Nungaray Héctor Miguel	2042781	IAS
Hernández Silva Carlos Yahir	1975208	ITS
Heredia López Betsy Aracely	2054413	IAS

Viernes 02 de Mayo del 2025

Monterrey, Nuevo León

Contenido

Introducción	2
Definición de Lenguaje Lógico	3
Características del Lenguaje Lógico.....	5
Ventajas y Desventajas	7
Aplicaciones del Lenguaje Lógico	8
Ejemplos	9
Conclusión General	12
Conclusiones Individuales.....	13
Referencias Bibliográficas	14

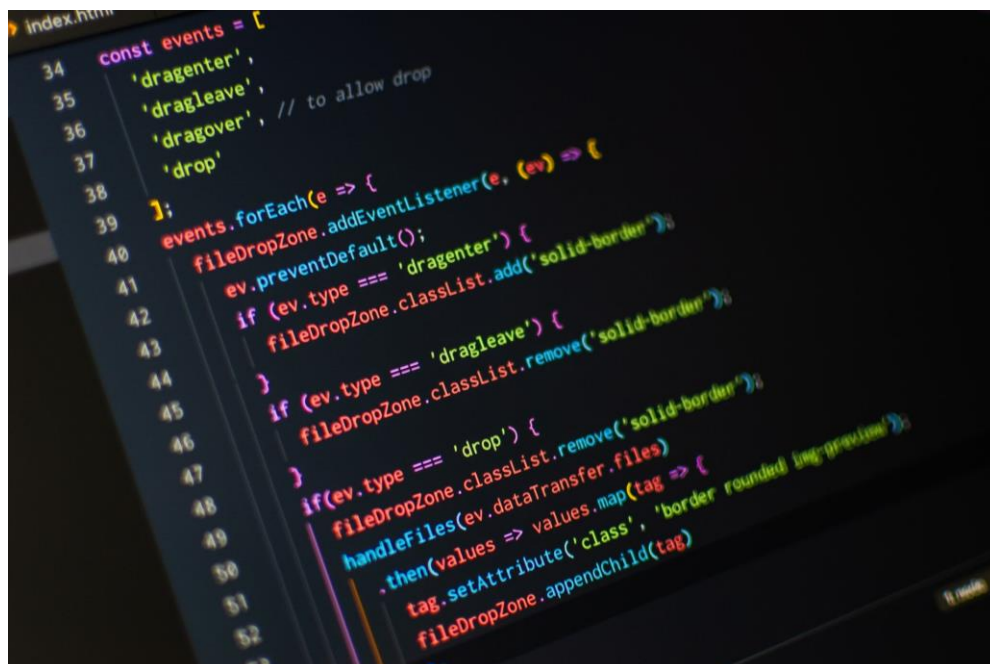
Introducción

El lenguaje lógico constituye un sistema formal de comunicación sustentado en símbolos, signos y reglas estrictas que permiten expresar afirmaciones, relaciones y razonamientos con una claridad y precisión imposible de alcanzar mediante el lenguaje natural. Esta formalización se basa en la teoría de lenguajes formales, donde cada expresión sigue una sintaxis rigurosa y su semántica está definida de manera inequívoca, eliminando cualquier posibilidad de interpretación subjetiva o ambigüedad.

Los orígenes del lenguaje lógico se remontan a la lógica matemática, especialmente a la lógica proposicional y de predicados, ramas que estudian las propiedades formales de las proposiciones y sus relaciones mediante operadores lógicos. En este contexto, cada símbolo (como $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$) tiene un significado exacto y operadores como la negación, conjunción o implicación permiten construir expresiones complejas que pueden analizarse y transformarse mediante reglas de inferencia predeterminadas.

A diferencia del lenguaje natural —en el cual las frases pueden admitir múltiples interpretaciones según el contexto—, el lenguaje lógico busca eliminar toda ambigüedad, garantizando que cada enunciado posea una única lectura posible. Esta característica resulta esencial para ámbitos como las matemáticas, la informática y la filosofía, donde la validez de un argumento o la corrección de una demostración dependen de la absoluta coherencia interna de las premisas y conclusiones.

En la práctica, los programas escritos en lenguajes lógicos (por ejemplo, Prolog) se componen de hechos y reglas, que el motor de inferencia utiliza para responder consultas mediante un proceso automático de unificación y backtracking. Este mecanismo permite al sistema explorar múltiples caminos de resolución de forma no determinista, retrocediendo cuando una rama no conduce a una solución y avanzando por otra, sin que el programador tenga que especificar manualmente los bucles o condiciones de control.



Definición de Lenguaje Lógico

El Lenguaje Lógico es un paradigma de programación declarativa basado en la lógica formal, donde los programas se describen como teorías lógicas y el cálculo se realiza mediante inferencia sobre esas teorías. En este enfoque se expresa qué se desea obtener, dejando al motor de ejecución la tarea de encontrar cómo lograrlo, lo que simplifica la especificación de problemas complejos. Además, su diseño lo hace especialmente idóneo para representar conocimiento estructurado y resolver consultas en sistemas de inteligencia artificial, bases de datos lógicas y sistemas expertos.

1. Basados en la lógica matemática

Los lenguajes lógicos se fundamentan en la lógica formal, principalmente en la lógica proposicional y de predicados, para representar hechos y relaciones mediante fórmulas bien definidas. Gracias a esta base matemática, cada expresión posee una interpretación única y acotada, lo que elimina ambigüedades y facilita el análisis riguroso de las proposiciones.

2. Programación declarativa

En lugar de especificar paso a paso cómo debe ejecutarse un algoritmo, el lenguaje lógico permite declarar la intención del programa —es decir, qué resultado se desea obtener— y el motor interno se encarga de descubrir el cómo mediante la búsqueda de soluciones. Esta separación entre especificación y control de flujo reduce la complejidad del código y permite una mayor abstracción en la resolución de problemas.

3. Uso de hechos y reglas

Los programas en lenguajes lógicos se estructuran mediante hechos (afirmaciones básicas sobre el dominio) y reglas (cláusulas que relacionan hechos mediante implicaciones lógicas) [Computer Science](#). Un ejemplo clásico en Prolog es:

prolog

CopiarEditar

padre(juan, maria).

abuelo(X, Y) :- padre(X, Z), padre(Z, Y).

donde padre/2 es un hecho y abuelo/2 una regla que usa unificación para inferir nuevas relaciones.

4. Resolución automática

El motor de inferencia analiza consultas transformándolas en objetivos que se satisfacen mediante la combinación de hechos y reglas, aplicando procedimientos de unificación y búsqueda en el espacio de

soluciones. Este proceso se realiza sin intervención del programador, quien no necesita especificar bucles o llamadas recursivas explícitas.

5. No determinismo controlado

Los lenguajes lógicos admiten la exploración de múltiples caminos de resolución, gestionando los llamados puntos de elección con técnicas como el backtracking, que retrocede y prueba alternativas hasta encontrar soluciones viables. Este mecanismo proporciona un control no determinista que, a pesar de su complejidad interna, resulta sencillo de usar para el programador.

6. Tipado dinámico

En la mayoría de los lenguajes lógicos, las variables no requieren una declaración de tipo previa; el sistema infiere los tipos de manera dinámica durante la ejecución, lo que agiliza el desarrollo y permite una mayor flexibilidad al manipular datos heterogéneos. Esta característica también simplifica la reutilización de predicados genéricos sobre diferentes dominios de datos.

7. Aplicaciones en IA y resolución de problemas

El lenguaje lógico es la columna vertebral de numerosos sistemas expertos, agentes inteligentes y motores de búsqueda semántica, donde la representación explícita de conocimiento y la inferencia automática son esenciales. Además, se emplea en planificación automática, verificación formal de programas y en lenguajes de consulta de bases de datos (por ejemplo, Datalog), gracias a su capacidad para expresar y razonar sobre relaciones complejas.



Características del Lenguaje Lógico

La programación lógica tiene varias características clave que la distinguen de otros paradigmas de programación. Estas son algunas de las más importantes:

1. **Uso de relaciones en lugar de instrucciones secuenciales:** A diferencia de la programación imperativa, que se basa en la ejecución secuencial de instrucciones, la programación lógica establece relaciones entre objetos o entidades, permitiendo que la computadora razone sobre estas relaciones para llegar a conclusiones o soluciones.
2. **Hechos y reglas:** Un programa lógico se compone principalmente de hechos y reglas. Los hechos son afirmaciones sobre el mundo (por ejemplo, "Rubí es un gato"), mientras que las reglas expresan relaciones lógicas entre hechos (por ejemplo, "todos los gatos tienen dientes"). Las reglas y hechos son la base para la deducción lógica y la inferencia.
3. **Lógica de primer orden:** La programación lógica se basa en la lógica de primer orden, que se utiliza para expresar relaciones y propiedades de los objetos en un dominio determinado. Este tipo de lógica permite formular preguntas y deducir respuestas utilizando inferencia lógica.
4. **Inferencia y deducción lógica:** En la programación lógica, la deducción se utiliza para derivar nuevas conclusiones a partir de hechos y reglas. Por ejemplo, si se sabe que "Rubí es un gato" y que "todos los gatos tienen dientes", se puede deducir que "Rubí tiene dientes".
5. **Sintaxis y semántica:** La programación lógica tiene una sintaxis formalizada, que permite expresar hechos, reglas y consultas en un formato comprensible por la máquina. La semántica describe cómo interpretar esas expresiones para determinar si una proposición es verdadera o falsa.
6. **Forma clausal:** La programación lógica se utiliza comúnmente con una forma clausal, en la que las sentencias se representan mediante cláusulas que contienen un encabezado y un cuerpo. Estas cláusulas se resuelven a través de un proceso de inferencia lógica.
7. **Modelo declarativo:** En lugar de decirle a la computadora cómo resolver un problema (como en la programación imperativa), la programación lógica se enfoca en qué objetivo se quiere lograr, dejando que el sistema deduzca cómo hacerlo a partir de las reglas y hechos definidos.
8. **Uso en inteligencia artificial:** La programación lógica se utiliza en inteligencia artificial, especialmente en sistemas que requieren razonamiento lógico, como en la representación del conocimiento y en la toma de decisiones basada en hechos y reglas.
9. **Consultas e interactividad:** Los programas lógicos permiten realizar consultas a la base de conocimiento. Por ejemplo, se puede preguntar "¿Quién es padre de Carlos?", y el sistema responderá basándose en las relaciones lógicas definidas en el programa.

Ventajas y Desventajas

Ventajas del lenguaje lógico

- **Enfoque declarativo:** Se indica qué se quiere lograr, no cómo, simplificando el diseño de programas.
- **Razonamiento automático:** Permite a la computadora deducir hechos nuevos a partir de hechos y reglas existentes.
- **Ideal para inteligencia artificial:** Facilita la representación y manejo del conocimiento lógico.
- **Basado en lógica formal:** Usa estructuras bien definidas (hechos, reglas, lógica de primer orden) que permiten resultados predecibles.
- **Consultas dinámicas:** Se pueden hacer preguntas al programa para obtener inferencias sin reescribir código.
- **Versatilidad de relaciones:** No requiere definir conceptos explícitos como "persona" para trabajar sobre ellos.

Desventajas del lenguaje lógico

- **Curva de aprendizaje alta:** Lenguajes como Prolog son muy diferentes de otros lenguajes tradicionales, lo que dificulta su comprensión.
- **Dificultad en la legibilidad:** Programas extensos pueden resultar complicados de entender debido a su estructura basada en hechos y reglas.
- **Menor control del flujo:** No existe una secuencia rígida de ejecución, lo que puede ser confuso en problemas que requieren pasos específicos.
- **Limitaciones prácticas:** Aunque poderoso para deducciones lógicas, no es siempre la mejor opción para tareas numéricas o procesamiento de datos intensivos.

Aplicaciones del Lenguaje Lógico

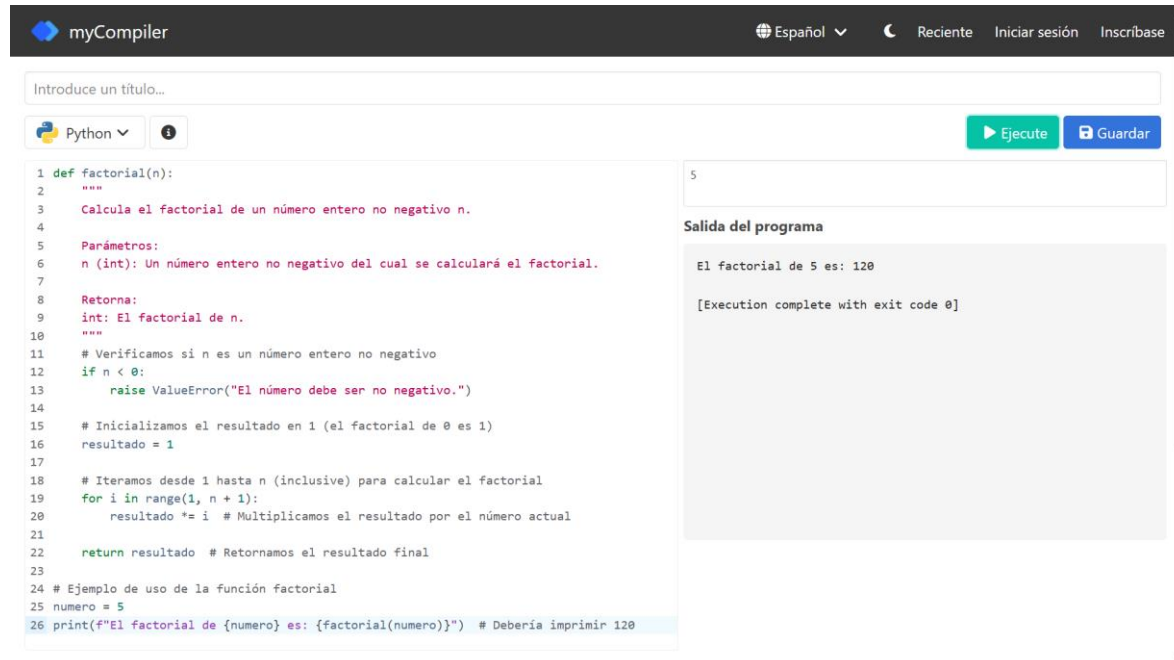
El lenguaje lógico se aplica principalmente en contextos donde la resolución de problemas requiere razonamiento simbólico, inferencia y toma de decisiones automatizada. Una de sus principales áreas de aplicación es en la inteligencia artificial (IA), especialmente en la programación de sistemas expertos. Estos sistemas simulan la toma de decisiones humanas mediante reglas lógicas, permitiendo que una computadora pueda inferir nuevas conclusiones a partir de un conjunto dado de hechos. Lenguajes como Prolog, uno de los más representativos del paradigma lógico, han sido ampliamente utilizados para construir motores de inferencia, asistentes inteligentes y sistemas de diagnóstico médico, gracias a su capacidad para representar relaciones complejas entre datos mediante hechos y reglas lógicas.

Otra aplicación significativa del lenguaje lógico se encuentra en el ámbito de la verificación formal de software y hardware, donde se utilizan métodos lógicos para demostrar la corrección de algoritmos o circuitos digitales respecto a sus especificaciones. Esto es especialmente importante en sistemas críticos como los aeroespaciales, automotrices o de control médico, donde los errores pueden tener consecuencias graves. Asimismo, el paradigma lógico también se aplica en bases de datos relacionales y en el análisis de conocimiento semántico dentro de la web, lo cual permite inferir significados, validar datos y establecer relaciones entre conceptos. Gracias a su precisión y claridad formal, el lenguaje lógico representa una herramienta poderosa para modelar procesos de razonamiento automatizado en campos donde la precisión y la deducción son fundamentales.



Ejemplos:

Ruedas Vazquez Jordan Emanuel



The screenshot shows the myCompiler web interface. At the top, there's a header with the logo, language (Español), theme (Reciente), and login/signup links. Below the header is a search bar and a dropdown menu set to Python. The main area is split into two panels. The left panel contains a Python code editor with a factorial function definition and its usage. The right panel shows the input '5' and the output 'El factorial de 5 es: 120'.

```
1 def factorial(n):
2     """
3     Calcula el factorial de un número entero no negativo n.
4
5     Parámetros:
6     n (int): Un número entero no negativo del cual se calculará el factorial.
7
8     Retorna:
9     int: El factorial de n.
10    """
11    # Verificamos si n es un número entero no negativo
12    if n < 0:
13        raise ValueError("El número debe ser no negativo.")
14
15    # Inicializamos el resultado en 1 (el factorial de 0 es 1)
16    resultado = 1
17
18    # Iteramos desde 1 hasta n (inclusive) para calcular el factorial
19    for i in range(1, n + 1):
20        resultado *= i # Multiplicamos el resultado por el número actual
21
22    return resultado # Retornamos el resultado final
23
24 # Ejemplo de uso de la función factorial
25 numero = 5
26 print(f"El factorial de {numero} es: {factorial(numero)}") # Debería imprimir 120
```

Salida del programa

El factorial de 5 es: 120

[Execution complete with exit code 0]

Datos de Entrada:

- `n (int)`: Un número entero no negativo del cual se desea calcular el factorial. Por ejemplo, si $n = 5$, se quiere calcular $5!$.

Proceso:

1. Validación: Se verifica si el número n es un entero no negativo. Si n es negativo, se lanza una excepción (error).
2. Inicialización: Se inicializa una variable llamada `resultado` en 1. Esto se debe a que el factorial de 0 es 1 y también sirve como punto de partida para la multiplicación.
3. Cálculo del Factorial:
 - Se utiliza un bucle que itera desde 1 hasta n (inclusive).
 - En cada iteración, se multiplica el valor actual de `resultado` por el número de la iteración (i).
 - Este proceso se repite hasta que se han multiplicado todos los números desde 1 hasta n .

Salida:

- `resultado (int)`: El valor del factorial de n . Por ejemplo, si $n = 5$, la salida será 120, ya que $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.

The screenshot shows a Python IDE with a file named 'Sin título0.py'. The code defines a function 'calcular_factorial' that calculates the factorial of a number. It includes comments in Spanish explaining the logic: checking for 0 or 1, initializing a result variable, using a loop to multiply numbers from 1 to the input, and printing the progress. The function is then called with the input 5. The output shows the step-by-step calculation: 1, 2, 6, 24, and finally 120.

```
1 def calcular_factorial(numero):
2     # Verificamos si el número es 0 o 1, ya que sus factoriales son 1
3     if numero <= 1:
4         return 1
5
6     # Inicializamos una variable para almacenar el resultado
7     resultado = 1
8
9     # Utilizamos un bucle para multiplicar todos los números desde 1 hasta el número
10    for i in range(1, numero + 1):
11        # Multiplicamos el resultado actual por el número del bucle
12        resultado *= i
13        # Mostramos el progreso del cálculo
14        print(f"Factorial parcial después de multiplicar por {i}: {resultado}")
15
16    # Devolvemos el resultado final
17    return resultado
18
19 # Solicitamos al usuario que ingrese un número
20 numero_usuario = int(input("Ingresa un número para calcular su factorial: "))
21
22 # Verificamos que el número sea positivo
23 if numero_usuario < 0:
24     # Informamos al usuario que no se puede calcular el factorial de números negativ
25     print("No se puede calcular el factorial de un número negativo.")
26 else:
27     # Llamamos a la función y guardamos el resultado
28     resultado_factorial = calcular_factorial(numero_usuario)
29     # Mostramos el resultado al usuario
30     print(f"El factorial de {numero_usuario} es: {resultado_factorial}")
31
```

Nombre	Tipo	Tamaño	Valor
numero_usuario	int	1	5
resultado_factorial	int	1	120

```
In [3]: %runfile "C:/Users/carla/OneDrive/Escritorio/Sin título0.py" --wdir
Ingresa un número para calcular su factorial: 5
Factorial parcial después de multiplicar por 1: 1
Factorial parcial después de multiplicar por 2: 2
Factorial parcial después de multiplicar por 3: 6
Factorial parcial después de multiplicar por 4: 24
Factorial parcial después de multiplicar por 5: 120
El factorial de 5 es: 120

In [4]:
```

```
def calcular_factorial(numero):
```

```
    # Verificamos si el número es 0 o 1, ya que sus factoriales son 1
```

```
    if numero <= 1:
```

```
        return 1
```

```
    # Inicializamos una variable para almacenar el resultado
```

```
    resultado = 1
```

```
    # Utilizamos un bucle para multiplicar todos los números desde 1 hasta el número dado
```

```
    for i in range(1, numero + 1):
```

```
        # Multiplicamos el resultado actual por el número del bucle
```

```
        resultado *= i
```

```
        # Mostramos el progreso del cálculo
```

```
        print(f"Factorial parcial después de multiplicar por {i}: {resultado}")
```

```
    # Devolvemos el resultado final
```

```
    return resultado
```

```
# Solicitamos al usuario que ingrese un número
```

```
numero_usuario = int(input("Ingresa un número para calcular su factorial: "))
```

```
# Verificamos que el número sea positivo

if numero_usuario < 0:

    # Informamos al usuario que no se puede calcular el factorial de números negativos

    print("No se puede calcular el factorial de un número negativo.")

else:

    # Llamamos a la función y guardamos el resultado

    resultado_factorial = calcular_factorial(numero_usuario)

    # Mostramos el resultado al usuario

    print(f"El factorial de {numero_usuario} es: {resultado_factorial}")
```

❖ Explicación del código

El programa consta de tres partes principales:

1. **Definición de la función `calcular_factorial`:** Esta función toma un número como entrada y calcula su factorial utilizando un bucle.
2. **Entrada del usuario:** El programa solicita al usuario que ingrese un número para calcular su factorial.
3. **Verificación y cálculo:** Comprueba si el número es válido (no negativo) y luego calcula y muestra el resultado.

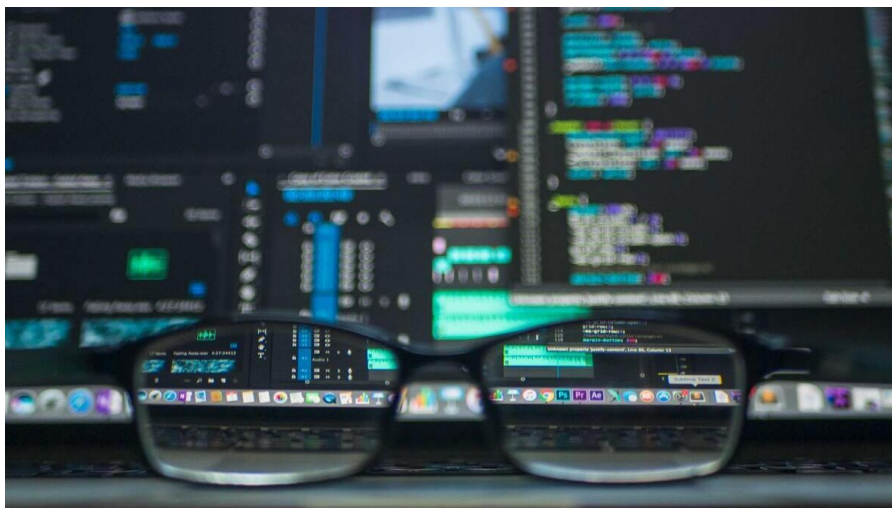
La lógica del factorial se implementa de manera iterativa, multiplicando cada número desde 1 hasta el número dado. El programa también muestra el progreso del cálculo para que puedas ver cómo va creciendo el factorial en cada paso.

Conclusión General

Al profundizar en el **lenguaje lógico**, el equipo identificó que este paradigma proporciona una estructura clara y precisa al pensamiento, gracias a su uso de símbolos y reglas formales que evitan las ambigüedades inherentes al lenguaje natural. Esta capacidad de **eliminar la confusión** radica en que cada expresión lógica está sujeta a una única interpretación, característica esencial para garantizar coherencia en el análisis de proposiciones y argumentos. Además, el lenguaje lógico facilita la **representación del conocimiento** en sistemas de inteligencia artificial y bases de datos, al permitir definir hechos y reglas de inferencia que máquinas pueden procesar de manera fiable.

El manejo del lenguaje lógico **fortalece el pensamiento crítico**, pues obliga a identificar premisas, establecer relaciones causales y evaluar conclusiones con estricta rigurosidad. Este ejercicio de razonamiento deductivo mejora la capacidad de **toma de decisiones** basada en evidencias y coherencia interna, tanto en contextos académicos como profesionales, donde la validez de un argumento puede tener consecuencias prácticas significativas. Asimismo, la práctica de la lógica promueve habilidades de **resolución de problemas** complejos, al descomponer situaciones en componentes atomizados y aplicar reglas de inferencia sistemáticas para generar soluciones.

Finalmente, el dominio del lenguaje lógico **trasciende el ámbito técnico**, pues los principios de claridad y precisión son igualmente valiosos en la comunicación cotidiana y en la formulación de ideas en cualquier disciplina. Aprender a expresarse con exactitud reduce malentendidos y favorece la colaboración interdisciplinar, ya que ofrece un marco común de interpretación. En consecuencia, la comprensión profunda de este lenguaje empodera al estudiante no solo para diseñar algoritmos o sistemas expertos, sino también para articular argumentos sólidos, persuadir con integridad y abordar desafíos de manera sistemática en la vida diaria.



Conclusiones Individuales

Ruedas Vazquez Jordan Emanuel

A través del estudio del lenguaje lógico, se ha logrado comprender la importancia de los paradigmas de programación que no se centran únicamente en la ejecución secuencial de instrucciones, sino en la representación del conocimiento y el razonamiento automático. Este enfoque promueve una forma diferente de estructurar soluciones, basada en hechos y reglas que permiten inferencias lógicas, lo cual resulta fundamental en campos como la inteligencia artificial, los sistemas expertos y la verificación formal de sistemas. Aprender sobre este paradigma permite desarrollar una visión más amplia y profunda de la programación, enriqueciendo el pensamiento computacional y fomentando la capacidad de resolver problemas complejos con métodos más abstractos y formales. En consecuencia, se reconoce el valor del lenguaje lógico no solo como una herramienta técnica, sino como una vía para potenciar el razonamiento estructurado y la comprensión avanzada de cómo una máquina puede simular procesos de pensamiento humano.

Enríquez Nungaray Héctor Miguel

Los lenguajes lógicos representan una poderosa herramienta en el ámbito de la programación declarativa, ya que permiten expresar el conocimiento y resolver problemas mediante hechos y reglas lógicas, sin necesidad de detallar el procedimiento paso a paso. Su enfoque basado en la lógica formal los hace especialmente útiles en campos como la inteligencia artificial, la resolución de problemas complejos y los sistemas expertos. Aunque no son tan utilizados en el desarrollo de software convencional, su capacidad para modelar razonamientos complejos y realizar inferencias automáticas los convierte en un recurso fundamental en aplicaciones que requieren inteligencia simbólica.

Hernández Silva Carlos Yahir

Los lenguajes lógicos nos ofrecen una manera diferente de programar: en lugar de decirle a la computadora paso por paso qué hacer, le enseñamos las relaciones entre las cosas y dejamos que ella misma saque conclusiones. Esto los hace especialmente valiosos en áreas como la inteligencia artificial, donde deducir y razonar es fundamental. Sin embargo, no todo es tan sencillo: aprender a pensar en términos de hechos y reglas puede ser complicado, y estos lenguajes no siempre son la mejor opción para resolver problemas que requieren cálculos o procesos muy específicos. Aun así, los lenguajes lógicos son una herramienta poderosa cuando se trata de trabajar con conocimiento y lógica de forma clara y estructurada.

Referencias Bibliográficas

Bratko, I. (2012). Prolog Programming for Artificial Intelligence (4th ed.). Harlow, England: Addison-Wesley.

Sterling, L., & Shapiro, E. (1994). The Art of Prolog: Advanced Programming Techniques (2nd ed.). Cambridge, MA: MIT Press.

Sebesta, R. W. (2012). Concepts of Programming Languages (10th ed.). Boston: Pearson Education.

Nilsson, U. & Maluszynski, J. (1995). Logic, Programming and Prolog (2nd ed.). Chichester: Wiley.

Clocksin, W. F., & Mellish, C. S. (2003). Programming in Prolog: Using the ISO Standard (5th ed.). Berlin: Springer.